

showfhir ZYENCODE vs quality tools — 2026-09-28

Encoder: C0RG ENCODER=ZYENCODE on showfhir

fhirdev: AUTO / PLUGIN (unchanged)

Routines added/changed: rehmp/C0RG/C0RGZYFX.m (new), rehmp/C0RG/C0RGZYEN.m (hook)

Executive summary

Stock YottaDB **ZYENCODE** is useful for encoder A/B benches against the C plugin, but its default dialect is **not FHIR/cds1-safe**. We built an MUMPS normalize layer (C0RGZYFX) so responses parse as JSON again. **Quality / CQL measurement may still not work perfectly** after that normalize — cds1 can fetch and build QDM elements, but CMS165 official-CQL reeval landed **0/0/0/0** where PLUGIN previously held **19/19/12/2**. Treat ZYENCODE+normalize as experimental for quality; keep **PLUGIN** (or JSNE) on hosts that must feed live measure calculation.

Part 1 — Baseline failure (before normalize)

Verdict then: not OK for quality tools

ZYENCODE responses were HTTP 200 but **not standard JSON/FHIR**:

- Leading dialect marker: body starts with 1{... → JSONDecodeError: Extra data

(dest root = chunk count; HTTP writers emit \$G(dest) before dest(1)...)

- Arrays encoded as 1-based objects ("entry": {"1": ...}) instead of JSON arrays
- Booleans often emitted as strings ("active": "true") when the M tree held bare "true" (ZYENCODE only emits JSON true/false for \$C(0)_"true|false")

Endpoint parse (strict json.loads) — before normalize

Path	HTTP	Strict JSON
/fhir/metadata	200	FAIL
/c0x/index/stat	200	FAIL
/c0x/presets	200	FAIL
/c0x/cohort?measure=CMS165v14	200	FAIL
/fhir?dfn=101124	200	FAIL
/fhir-quality-report?measure=CMS165v14	200	PASS (different encoder path)

cds1 rejection (verbatim)

POST https://cds1.vistaplex.org/quality/evaluate-cohort with

fhirBase=https://showfhir.vistaplex.org/fhir, patients:["101124"]:

```
{
  "status": "error",
  "message": "Invalid JSON from https://showfhir.vistaplex.org/fhir?dfn=101124"
}
```

Code path (cds-hooks-on-fhir/services/quality-eval/server.js):

evaluateCohort → fetchPatientBundle(fhirBase, dfn)

GET {fhirBase}?dfn={dfn}

fetchJson → JSON.parse(body)

On failure: Invalid JSON from \${urlString} (lines 114-117)

Node: SyntaxError: Unexpected non-whitespace character after JSON at position 1

Body started 1{"entry":... (hex 31 7b).

Even after stripping the leading 1, entry was still an **object** ("1"... "N"), so (bundle.entry || []).map(...) in fhir-to-qdm-patient.js would fail next.

Part 2 — MUMPS normalize layer (what we added)

Design

Problem	Fix	
1-based M lists → {"1":...}	Before encode: reindex contiguous 1..n children to 0..n-1 so ZYENCODE emits [...]	
Leading 1 before {	After encode: clear numeric dest root (chunk count)	
1 MiB chunk → FORCESTR MAXSTRLEN	Split dest pieces to ≤64 KiB	
"active": "true" strings	Map bare "true"/"false" leaves to `\$(0)_true` false" before encode	

Hook in ENCODEZY^CORGZYEN

```
ENCODEZY(XUR00T,XUJSON,XUERR) ;
  N C0FARG,C0FN,C0RZYB
  ...
  K @XUJSON
  ; Work copy: reindex 1-based lists → 0-based so ZYENCODE emits JSON arrays.
  K C0RZYB M C0RZYB=@XUR00T
  I $(T(PREP^C0RGZYFX)'='') D PREP^C0RGZYFX("C0RZYB")
  N $ET,$ES
  S $ET="G ZYENCFAIL"
  S C0FARG=XUJSON_"=C0RZYB"
  ZYENCODE @C0FARG
  S $ET=""
  S C0FN=+$G(@XUJSON)
  I C0FN<1 D FAIL^C0RGFENC(XUERR,"zyencode-empty") Q
  ; Drop count root so clients do not see a leading "1" before '{'.
```

```

I $T(FHIRIFY^C0RGZYFX)'="" D FHIRIFY^C0RGZYFX(XUJJSON)
; Keep pieces under MAXSTLEN for FORCESTR^C0FHIRBU / HTTP workers.
I $T(SPLIT^C0RGZYFX)'="" D SPLIT^C0RGZYFX(XUJJSON,65536)
D SETPATH^C0RGFENC("ZYENCOD", "native+0base")
Q

```

New routine C0RGZYFX (core)

```

C0RGZYFX ;C0RG/WorldVistA - ZYENCOD FHIR-array normalize ;Sep 28, 2026
;
PREP(XUR00T) ; Reindex 1-based lists under XUR00T to 0-based (in place)
I '$L($G(XUR00T)) Q
I '$D(@XUR00T) Q
D WALK(XUR00T)
Q
;
WALK(NA) ; Recurse, fix boolean strings, then shift 1..n lists to 0-based
N K,V
; Bare "true"/"false" → XLFJSON boolean markers for ZYENCOD
I $D(@NA)#2 D
. S V=$G(@NA)
. I V="true" S @NA=$C(0)"true"
. I V="false" S @NA=$C(0)"false"
S K=""
F S K=$O(@NA@(K)) Q:K="" D
. I $D(@NA@(K))>9 D WALK($NA(@NA@(K)))
. E I $D(@NA@(K))#2 D
. . S V=$G(@NA@(K))
. . I V="true" S @NA@(K)=$C(0)"true"
. . I V="false" S @NA@(K)=$C(0)"false"
I $$ISLIST(NA) D SHIFT0(NA)
Q
;
ISLIST(NA) ; $$ 1 if immediate children are exactly 1,2,..n
N K,N,EXPECT
S K=$O(@NA@(""))
I K'=1 Q 0
S EXPECT=1,N=0
F Q:K="" D Q:EXPECT<1
. I K'=EXPECT S EXPECT=0 Q
. S N=N+1,EXPECT=EXPECT+1
. S K=$O(@NA@(K))
I EXPECT<1 Q 0
I N<1 Q 0
Q 1
;
SHIFT0(NA) ; Move @(1)..@(n) → @(0)..@(n-1)
N C0FZ,I,N
K C0FZ
S N=0,I=0
F S I=$O(@NA@(I)) Q:'I S N=N+1 M C0FZ(I-1)=@NA@(I)
K @NA
M @NA=C0FZ
Q
;
FHIRIFY(XUJJSON) ; Drop chunk-count root value
I '$L($G(XUJJSON)) Q
I $D(@XUJJSON)#2 S @XUJJSON=""
Q
;
SPLIT(XUJJSON,MAX) ; Break oversized dest chunks (default 64KiB)

```

```

N C0FI,C0FO,C0FS,C0FP,C0FL,C0FN,C0FM
I '$L($G(XUJSON)) Q
S MAX=+$G(MAX) I MAX<4096 S MAX=65536
K C0FO S C0FN=0,C0FI=0
F S C0FI=$0(@XUJSON@(C0FI)) Q:'C0FI D
. S C0FS=$G(@XUJSON@(C0FI)),C0FL=$L(C0FS)
. I C0FL '>MAX S C0FN=C0FN+1,C0FO(C0FN)=C0FS Q
. S C0FP=1
. F Q:C0FP>C0FL D
. . S C0FM=$S((C0FP+MAX-1)<C0FL:MAX,1:C0FL-C0FP+1)
. . S C0FN=C0FN+1,C0FO(C0FN)=$E(C0FS,C0FP,C0FP+C0FM-1)
. . S C0FP=C0FP+C0FM
K @XUJSON
M @XUJSON=C0FO
Q

```

Source of truth in repo: rehmp/C0RG/C0RGZYFX.m and the ENCODEZY hook in rehmp/C0RG/C0RGZYEN.m.

Part 3 — After normalize (quality still may not be perfect)

What improved

Check	After C0RGZYFX
/fhir?dfn=101124	HTTP 200, strict JSON, entry is an array
Large DFNs (101070, 101083)	HTTP 200 (chunk split avoids FORCESTR MAXSTRLEN)
Booleans	e.g. "active": true (170 real JSON bools / 0 string bools on DFN 101070)
cds1 JSON.parse	No longer Invalid JSON from ...
cds1 QDM bridge	Builds elements (e.g. 881 / 587 / 46 on sample DFNs)

What still fails or is unreliable for quality measurement

Check	Result	Note
CMS165 POST /fhir-quality-reeval	status done , SUM 0/0/0/0	Under PLUGIN this host previously held 19/19/12/2
Sample cds1 evaluate-cohort on 101070/101083/101124	ipp/denom/numer/denex all 0 despite hundreds of QDM elements	Parse works; official CQL population gates do not match as they did on PLUGIN JSON

Bottom line for quality tools: the normalize layer makes ZYENCODE **parseable** and usable for telemetry / A/B of encode shape, but **it might not work perfectly with quality calculations**. Remaining YottaDB/ZYENCODE dialect drift (beyond arrays and booleans — e.g. typing, empty nodes, UTF-8, other XLFJSON conventions) can still zero out measure rates. Prefer **PLUGIN** on any server expected to drive cds1 CQL / dashboards.

Implication (unchanged policy)

- **Bench / comparison:** showfhir on ZYENCODE (+C0RGZYFX) vs fhirdev on PLUGIN is fine.
- **Live quality measurement:** keep PLUGIN (or JSNE). Do not treat ZYENCODE normalize as production parity for CQL.